

Incoming Request Processing Workflow

An AI prototype that classifies incoming customer requests and executes a distinct, multi-step remediation workflow for each type.



The Problem

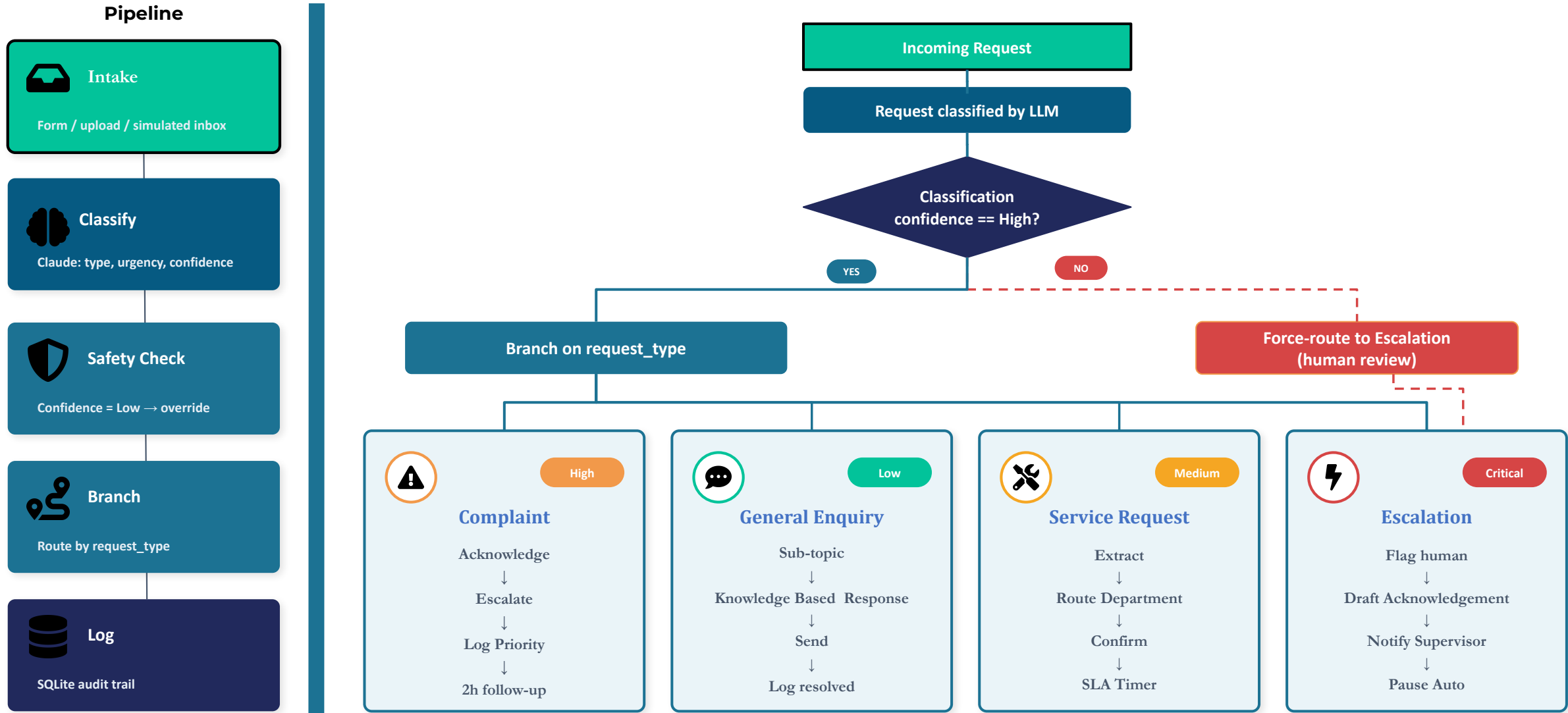
- Every incoming request - complaint, enquiry, or service issue - is read and routed by hand, so speed and consistency depend on whoever picks it up
- Urgent cases sit in the same queue as routine questions, so escalations get noticed late
- Skilled agents spend their time working out what a message is, instead of resolving it



The Objective

- Classify every request by type and urgency the moment it arrives
- Trigger the right remediation path automatically: draft the reply, route to the right team, set the follow-up
- Keep humans firmly in control of critical and uncertain cases
- Give operations leaders live visibility: volumes, case status, and measurable time and cost saved

Decision logic: confidence gates the override, then branch by type



Dashed line: the low-confidence override and a direct "Escalation" classification both land on the same remediation branch.

Built on Claude classification + a confidence-gated override

 workflows.py – run_workflow()

```
if confidence == "Low" and \
    request_type != "Escalation":
    result = handle_escalation(
        request_id, text, classification
    )
    result["override_applied"] = True
    result["override_reason"] = (
        f"Low confidence on '{request_type}' "
        "-> auto-escalated to human review"
    )
    return result

handler = BRANCH_MAP.get(
    request_type, handle_general_enquiry
)
return handler(request_id, text, classification)
```

Tech Stack

- ✓ **Python 3** — orchestration: classifier.py, workflows.py, storage.py
- ✓ **Claude API (claude-sonnet-4-5)** — classification + draft generation
- ✓ **Streamlit** — 3-tab UI: intake, batch run, dashboard
- ✓ **SQLite + Pandas** — audit trail, dashboard aggregation, CSV export

Branching & Reasoning Design

- Classifier self-reports confidence, not just a label — used directly as the override trigger
- Department routing inferred by keyword match (billing / technical / account / shipping / general)
- Offline keyword-fallback engine mirrors the same branch/step contract — pipeline never goes down

What broke, what we watched out for, and what it taught us



Bug: over-eager override

The offline fallback flagged every signal-free message as low-confidence, sending all enquiries to review. Fixed by limiting that to short, vague, question-mark-free text.



Trade-off: draft-then-hold for Escalation

Critical cases get a drafted reply that is held, not auto-sent - a human always reviews it before an upset customer sees it.



Trade-off: keyword department routing

Keyword-matched routing is fast and fully explainable to an ops team, but coarser than letting the model pick the department directly.



Challenge: overreliance on the LLM

Trusting one model for both classification and drafting is a single point of failure. Countered with the confidence override, draft-then-hold, and an offline fallback.



Challenge: no way to quantify output quality

No accuracy metric exists beyond manually reading case traces. A labelled evaluation set with a confusion matrix is the top v2 priority.



Learning: the hard part isn't the build

Wiring the pipeline was the easy part. The real work was tailoring it to real pain points - the PRD was built by walking through the agent, supervisor, and admin perspectives first.

Classified, remediated, reviewed by a human, and audited — end to end

4

Remediation branches

70%

Auto-handled (deflection)



Human review queue built-in



Live email alerts to stakeholders

Sample Run — Volume by Type



Next Steps

- Labelled evaluation set to measure classification accuracy; model-driven department routing
- Case threading — link repeat contacts from the same customer and auto-escalate
- Urgency-scaled SLAs and follow-ups; Slack alerts and ticketing integration (e.g. Zendesk)
- Feedback loop: learn from human reclassifications to refine the classifier prompt